any “hole”, this could be expressed as well as the range between 0 and `max(clusterIndices)`.

#4 Cycle through all coordinates (assuming `dim` , the number of coordinates, is f.i. a class variable, otherwise you could use `|centroids[i]|`)

#5 Each centroid’s coordinate is computed as the mean of the corresponding coordinate of all points assigned to it.

#6 return the centroids.

To update centroids, as we mentioned a few times, we will just compute the center of mass for each cluster: this means that we need to group all points by the centroid they are assigned to, and then for each group compute the mean of the points’ coordinates.

This pseudocode implementation doesn’t apply any correction to remove centroids that has no point assigned to them: they are just ignored, as well as it is not tackled the issue with the centroids array, that is simply initialized to the empty one, assuming it will be re-sized dynamically while adding new elements to it. But of course, in a real implementation, both issues should be taken care for, according to what each language allows for initialization and re-sizing of arrays.

If you’d like to take a look at real implementations of k-means, you can find a Python version on the book’s repo; staying on GitHub, at https://github.com/andreaferretti/kmeans you can also find a nice resource (unrelated to the book) that, like a Rosetta stone, contains implementations of this algorithm in many programming languages (you can find a version for most of the mainstream languages).